23. Ejercicio de Skill Camaleón

Jonathan Ricardo Serna Galina
20074902

Programación Asistida

1. Descripción

En este ejercicio creé, con apoyo de la IA, una skill que cambia el estilo de los comentarios y mensajes de un programa sin alterar su funcionamiento. Elegí un tono de carta medieval: solemne, cercano y reflexivo. Tomé como referencia el ambiente de la carta de Ezio para su hermana, pero utilicé frases originales.

Guardé las instrucciones en SKILL.md, dentro de la carpeta del ejercicio 23. Le indiqué al agente que conservara el idioma de cada texto y que solo adaptara lo que una persona lee. La demostración está en JavaScript y no requiere paquetes externos.

2. Ejemplos antes y después

En la salida original obtuve:

```
Proceso iniciado.
Total: 25
```

Después de aplicar la skill, obtuve:

```
Doy comienzo a esta jornada.
Que conste en esta crónica el total: 25
```

También cambié el mensaje de error a: "No me fue posible llevar esta jornada a término". El error sigue comunicando un fallo. En el comentario conservé el inglés original y adapté su tono.

La expresión ${total}, el valor de estado "ready" y la ruta "./records/journey.json" permanecieron iguales. Así separé el texto visible de las cadenas que usa el programa.

3. Cómo la utilicé

Le pedí al agente que usara medieval-chameleon sobre demo.mjs, cambiara únicamente comentarios y mensajes, y verificara que el código quedara intacto. Conservé demo.before.mjs como referencia para comparar.

```
node demo.before.mjs
node demo.mjs
```

4. Criterios que seguí

Solo permití cambios en comentarios explicativos y texto dirigido a personas. Conservé nombres, operaciones, condiciones, llamadas y datos del programa. También protegí las expresiones insertadas en los mensajes, las rutas y los valores utilizados en comparaciones.

Le indiqué a la skill que revisara el archivo original antes de editar y comparara el resultado después. Si una transformación cambia código, debe deshacer sus propios cambios con /undo cuando esté disponible, o restaurarlos desde el original retenido. Después debe mejorar la skill antes de repetir el intento.

No corregí la lógica del programa para hacer pasar la prueba. La transformación de la demostración solo modificó cuatro fragmentos de texto y se mantuvo dentro de la carpeta del ejercicio.

5. Cómo comprobé el resultado

Con ayuda del agente, comprobé la sintaxis de ambos archivos con node --check. Al revertir en memoria las cuatro sustituciones de texto, el contenido restante coincidió exactamente con el original.

También comparé la ejecución de ambas versiones y simulé en memoria las ramas de éxito y error. El total siguió siendo 25. Se conservaron los datos exportados, la ruta y la cantidad de mensajes de cada tipo. Solo cambió la redacción prevista.

Revisé la estructura de la skill con Node.js. El validador oficial no pudo ejecutarse porque faltaba PyYAML, por lo que no lo presento como una comprobación completada.

6. Lo que aprendí

Aprendí que una skill necesita límites claros. Aunque dos elementos sean cadenas de texto, uno puede ser un mensaje y otro un dato necesario para el programa. Por eso comprobé qué podía cambiar antes de aplicar el tema medieval.

Mi resultado fue una skill reutilizable y una demostración con texto adaptado. Pude cambiar la forma de comunicar el resultado y conservar el mismo comportamiento.

Archivos que respaldan mi explicación

En el ejercicio 23: .agents/skills/medieval-chameleon/SKILL.md, demo.before.mjs, demo.mjs y README.md.